UNIDAD DIDÁCTICA 3

Tema 3 · JavaScript avanzado

Funciones, colecciones y organización del código

Desarrollo Web en Entorno Cliente · 2.º DAW
Curso 2026/2027 · Antonio Martín León

Aprender a transformar datos y escribir funciones reutilizables antes de trabajar con eventos y DOM.

Continuidad con el tema 2: variables, condiciones, bucles, arrays y objetos básicos. Ejemplos ejecutables en consola y una práctica final organizada en módulos.

Recorrido de la unidad

1. Valores por defecto: ?? y ??=.
2. Funciones flecha y callbacks.
3. Parámetros rest y sintaxis spread.
4. Desestructuración de arrays y objetos.
5. Métodos de arrays y transformación de objetos.
6. Importación y exportación de módulos.

Trabajaremos sobre todo funciones, arrays y estructuras propias del RA4, y reforzaremos herramientas predefinidas del lenguaje relacionadas con el RA3. Los objetos del navegador y otros criterios de esos RA se completan en otras unidades.

Cómo ejecutar los ejemplos

index.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Prácticas del tema 3</title>
  <script src="app.js" defer></script>
</head>
<body>
  <h1>Abre la consola del navegador</h1>
</body>
</html>
```

Crea app.js en la misma carpeta. Abre el HTML con Live Server y consulta la consola del navegador.

Cada página contiene un ejemplo independiente, salvo que se indique continuación. Sustituye el contenido de app.js entre ejemplos para no redeclarar constantes. En el bloque de módulos cambiaremos el script a type="module".

Un valor por defecto con ??

a ?? b devuelve b únicamente cuando a es null o undefined. Si a contiene 0, false o una cadena vacía, conserva ese valor.

JavaScript

```
const descuento = 0;
const nombre = null;
console.log(descuento ?? 10); // 0
console.log(nombre ?? "Invitado"); // "Invitado"

console.log(0 || 10);     // 10
console.log(false ?? true); // false
console.log("" ?? "Sin texto"); // ""
```

|| utiliza el segundo valor ante cualquier valor falsy, como 0, false, "", null, undefined o NaN. ?? no sustituye NaN. Elige según lo que signifique “dato ausente” en tu problema.

Asignación condicional con ??=

x ??= valor asigna el valor solo si x es null o undefined. Resulta útil para inicializar datos que todavía no existen.

JavaScript

```
const preferencias = { volumen: 0 };
preferencias.volumen ??= 50;
preferencias.tema ??= "claro";
console.log(preferencias); // { volumen: 0, tema: "claro" }

let total;
total ??= 0;
total += 5;
console.log(total); // 5
```

const impide reasignar la variable preferencias, pero permite modificar sus propiedades. No mezcles ?? directamente con || o &&: usa paréntesis y decide qué operación debe evaluarse primero.

Funciones flecha: la sintaxis

JavaScript

```
const sumar = (a, b) => a + b;
const doble = numero => numero * 2;
const saludar = () => "Hola";

const calcularImporte = (precio, unidades) => {
  const total = precio * unidades;
  return total;
};

console.log(sumar(2, 3)); // 5
console.log(doble(4)); // 8
console.log(saludar()); // "Hola"
console.log(calcularImporte(5, 3)); // 15
```

Una expresión sin llaves se devuelve de forma implícita. Si usas un bloque con llaves, escribe return para devolver un resultado. Con cero o varios parámetros, los paréntesis son obligatorios.

Retorno de objetos y uso de this

```javascript
const crearPersona = nombre => ({ nombre, activo: true });
console.log(crearPersona("Ana"));
// { nombre: "Ana", activo: true }

const persona = {
  nombre: "Ana",
  saludar() {
    return `Hola, soy ${this.nombre}`;
  }
};
console.log(persona.saludar()); // "Hola, soy Ana"
```

Para devolver un objeto directamente desde una flecha, rodéalo con paréntesis. Así las llaves se interpretan como un objeto y no como el cuerpo de la función.

Las flechas no tienen su propio this: lo toman del contexto exterior. No sustituyas automáticamente todos los métodos por flechas. Para un método que usa this como el objeto receptor, utiliza la sintaxis del ejemplo.

Callbacks: funciones recibidas como datos

Una callback es una función que se pasa a otra para que esta la ejecute. No tiene que ser anónima, ni flecha, ni asíncrona.

JavaScript

```
function aplicar(numero, operacion) {
  return operacion(numero);
}
const doble = n => n * 2;

console.log(aplicar(4, doble)); // 8
console.log(aplicar(4, n => n + 3)); // 7
```

aplicar recibe el dato y la operación. La misma función puede producir resultados distintos según la callback que le pasemos.

Pasamos doble, no doble(). Una callback no convierte por sí sola un programa en programación funcional. Aquí todas las operaciones se ejecutan de forma síncrona.

Parámetros rest: reunir argumentos

JavaScript

```
function sumar(...numeros) {
  let total = 0;
  for (const numero of numeros) total += numero;
  return total;
}
console.log(sumar(2, 3, 4)); // 9
console.log(sumar()); // 0

const presentar = (nombre, ...aficiones) => {
  return `${nombre}: ${aficiones.join(", ")}`;
};
console.log(presentar("Ana", "cine", "música"));
```

Rest reúne argumentos en un array real. Debe ser el último parámetro y solo puede haber uno. En este ejemplo, numeros recibe todos los argumentos y aficiones recibe los que aparecen después del nombre.

Spread: expandir valores

```javascript
const notas = [5, 8, 6];
console.log(Math.max(...notas)); // 8

const masNotas = [...notas, 9];
console.log(masNotas); // [5, 8, 6, 9]

const base = { nombre: "Ana", activo: false };
const actualizado = { ...base, activo: true };
console.log(base.activo); // false
console.log(actualizado.activo); // true
```

En una llamada o un array, spread expande un iterable. En un objeto, copia sus propiedades propias enumerables. Si se repite una propiedad, prevalece la última.

Rest reúne; spread expande. Comparten los tres puntos, pero cumplen funciones diferentes. Math.max(...[]) devuelve -Infinity: controla el caso vacío si buscas un máximo significativo.

Copiar no siempre significa independizar

```javascript
const original = {
  nombre: "Ana",
  direccion: { ciudad: "Granada" }
};
const copia = { ...original };
copia.nombre = "Luis";
copia.direccion.ciudad = "Málaga";

console.log(original.nombre); // "Ana"
console.log(original.direccion.ciudad); // "Málaga"
```

Spread realiza una copia superficial. El objeto exterior es nuevo, pero los objetos anidados siguen compartidos.

Para modificar una rama sin afectar al original, copia también esa rama: { ...original, direccion: { ...original.direccion, ciudad: "Sevilla" } }. Lo mismo ocurre al copiar arrays que contienen objetos.

Desestructuración de arrays

JavaScript

```
const notas = [8, 6, 9, 5];
const [primera, segunda, ...resto] = notas;
console.log(primera, segunda); // 8 6
console.log(resto); // [9, 5]

const [valor = 10] = [];
console.log(valor); // 10

let a = 1;
let b = 2;
[a, b] = [b, a];
console.log(a, b); // 2 1
```

En arrays importa la posición. Puedes omitir posiciones con comas. Los valores por defecto se aplican a undefined, no a null. Desestructurar facilita leer el código; no garantiza una mejora de rendimiento ni una copia profunda.

Desestructuración de objetos

```javascript
const persona = { nombre: "Ana", edad: 24, ciudad: "Granada" };
const { edad, nombre: nombreAlumno, activo = true } = persona;
console.log(nombreAlumno, edad, activo); // "Ana" 24 true

const { ciudad, ...otrosDatos } = persona;
console.log(ciudad); // "Granada"
console.log(otrosDatos); // { nombre: "Ana", edad: 24 }

const describir = ({ nombre, edad }) => `${nombre}: ${edad}`;
console.log(describir(persona)); // "Ana: 24"
```

En objetos importan los nombres de las propiedades, no el orden. nombre: nombreAlumno cambia el nombre de la variable creada. La última función desestructura el objeto recibido como parámetro.

Desestructuración anidada

JavaScript

```
const curso = {
  titulo: "JavaScript",
  docente: { nombre: "Antonio" },
  grupos: ["A", "B"]
};
const {
  titulo,
  docente: { nombre },
  grupos: [primerGrupo]
} = curso;
console.log(titulo, nombre, primerGrupo);
// "JavaScript" "Antonio" "A"
```

El patrón sigue la estructura de los datos. Aquí se crean titulo, nombre y primerGrupo, no una variable docente. Si un objeto intermedio es null o undefined, este patrón falla: comprueba los datos antes de acceder a estructuras opcionales.

Objeto JavaScript y texto JSON

```javascript
const persona = { nombre: "Ana", edad: 24 };
const texto = JSON.stringify(persona);
console.log(typeof persona); // "object"
console.log(typeof texto); // "string"
console.log(texto); // {"nombre":"Ana","edad":24}

const recuperada = JSON.parse(texto);
console.log(recuperada.nombre); // "Ana"
```

Un objeto es un valor en memoria. JSON es un formato textual de intercambio. Un array de objetos no es “un array de JSON”.

En JSON, las claves y cadenas usan comillas dobles; no se admiten funciones, comentarios ni undefined. JSON.parse falla si el texto no tiene formato válido. Aquí usamos un texto conocido y correcto.

Recorrer propiedades de un objeto

JavaScript

```
const producto = { nombre: "Libro", precio: 20 };
producto.stock = 3;
producto["precio"] = 18;

for (const clave in producto) {
  if (Object.hasOwn(producto, clave)) {
    console.log(clave, producto[clave]);
  }
}
delete producto.stock;
console.log(producto); // { nombre: "Libro", precio: 18 }
```

for...in recorre claves enumerables, incluidas las heredadas. Object.hasOwn permite limitar el recorrido a las propias.

Para recorrer los valores de un array, utiliza for...of o sus métodos. Para trabajar con claves, valores o pares de un objeto, veremos Object.keys, Object.values y Object.entries.

forEach: recorrer sin crear otro array

```javascript
const productos = [
  { nombre: "Libro", precio: 20 },
  { nombre: "Lápiz", precio: 2 }
];
productos.forEach((producto, indice) => {
  console.log(`${indice + 1}. ${producto.nombre}`);
});
// 1. Libro
// 2. Lápiz
```

La callback recibe el elemento, el índice y el array. Puedes declarar solo los parámetros que necesites. forEach devuelve undefined.

Utilízalo para realizar una acción por elemento. No devuelve un array transformado y no admite break para detener el recorrido. Para transformar, usa map; para interrumpir un recorrido general, usa un bucle.

map y filter: transformar o seleccionar

```JavaScript
const precios = [10, 20, 30];
const dobles = precios.map(precio => precio * 2);
const baratos = precios.filter(precio => precio <= 20);
console.log(dobles); // [20, 40, 60]
console.log(baratos); // [10, 20]
console.log(precios); // [10, 20, 30]

const incorrecto = precios.map(precio => { precio * 2; });
console.log(incorrecto); // [undefined, undefined, undefined]
```

map obtiene un resultado por elemento; filter conserva los elementos que cumplen una condición. Ambos crean un array nuevo.

En el último ejemplo falta return. Estos métodos no modifican por sí mismos el array original, pero una callback puede hacerlo. Los objetos conservados por filter siguen siendo los mismos objetos, no copias.

find y findIndex: localizar una coincidencia

JavaScript

```
const productos = [
  { id: 1, nombre: "Libro" },
  { id: 2, nombre: "Lápiz" }
];
const encontrado = productos.find(p => p.id === 2);
const posicion = productos.findIndex(p => p.id === 2);
console.log(encontrado.nombre); // "Lápiz"
console.log(posicion); // 1
console.log(productos.find(p => p.id === 9)); // undefined
console.log(productos.findIndex(p => p.id === 9)); // -1
```

find devuelve el primer elemento; findIndex, su posición. Comprueba si existe antes de acceder a sus propiedades. No uses if (posicion) para saber si se encontró: 0 es una posición válida y -1 es truthy.

some, every y reduce

JavaScript

```
const notas = [4, 7, 9];
console.log(notas.some(n => n >= 5)); // true
console.log(notas.every(n => n >= 5)); // false

const total = notas.reduce((acumulado, nota) => {
  return acumulado + nota;
}, 0);
console.log(total); // 20
console.log([].reduce((suma, n) => suma + n, 0)); // 0
```

some comprueba si al menos un elemento cumple la condición. every comprueba si la cumplen todos. reduce acumula un resultado: aquí empieza en 0 y pasa por 4, 11 y 20.

En un array vacío, some devuelve false y every devuelve true. Si quieres exigir al menos un dato: notas.length > 0 && notas.every(...). reduce sin valor inicial falla cuando el array está vacío.

Ordenar sin alterar el original

```javascript
const numeros = [10, 2, 30];
const ordenados = [...numeros].sort((a, b) => a - b);
console.log(ordenados); // [2, 10, 30]
console.log(numeros); // [10, 2, 30]

const productos = [
  { nombre: "Libro", precio: 20 },
  { nombre: "Lápiz", precio: 2 }
];
const porPrecio = [...productos].sort((a, b) => a.precio - b.precio);
console.log(porPrecio.map(p => p.nombre)); // ["Lápiz", "Libro"]
```

sort modifica el array donde se aplica. Sin comparador ordena según representaciones de texto: no sirve para ordenar números como esperamos. Para texto: a.nombre.localeCompare(b.nombre, "es"). La copia del array sigue compartiendo los objetos.

Eliminar valores repetidos

JavaScript

```
const numeros = [2, 4, 2, 6, 4];
const unicos = numeros.filter((numero, indice, array) => {
  return array.indexOf(numero) === indice;
});
console.log(unicos); // [2, 4, 6]

const personas = [{ id: 1 }, { id: 2 }, { id: 1 }];
const sinRepetir = personas.filter((persona, indice, array) => {
  return array.findIndex(p => p.id === persona.id) === indice;
});
console.log(sinRepetir); // [{ id: 1 }, { id: 2 }]
```

Se conserva la primera aparición. Para objetos, decide qué propiedad define una repetición: aquí, id. El primer patrón usa números ordinarios; indexOf no encuentra NaN. Estos recorridos anidados pueden ser costosos con colecciones grandes.

Mezclar y seleccionar al azar

JavaScript

```
function mezclar(datos) {
  const copia = [...datos];
  for (let i = copia.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copia[i], copia[j]] = [copia[j], copia[i]];
  }
  return copia;
}
console.log(mezclar([1, 2, 3, 4])); // El orden puede variar
const muestra = [1, 2, 3, 4].filter(() => Math.random() < 0.5);
console.log(muestra); // La cantidad puede variar
```

La mezcla aplica Fisher-Yates a una copia. No uses sort(() => Math.random() - 0.5): incumple las reglas del comparador y puede sesgar el resultado. El filtro selecciona cada elemento con probabilidad aproximada 1/2; no garantiza conservar la mitad.

Object.keys, values y entries

JavaScript

```
const puntos = { Ana: 8, Luis: 6 };
console.log(Object.keys(puntos)); // ["Ana", "Luis"]
console.log(Object.values(puntos)); // [8, 6]
console.log(Object.entries(puntos)); // [["Ana", 8], ["Luis", 6]]

for (const [nombre, nota] of Object.entries(puntos)) {
  console.log(`${nombre}: ${nota}`);
}
const total = Object.values(puntos).reduce((suma, n) => suma + n, 0);
console.log(total); // 14
```

Estos métodos obtienen arrays a partir de las propiedades propias enumerables con clave de texto. Un objeto ordinario no tiene forEach ni get: usa puntos[nombre], no puntos.get(nombre). Object y Map son estructuras diferentes.

Object.fromEntries: volver a un objeto

```javascript
const precios = { libro: 20, lapiz: 2 };
const pares = Object.entries(precios);
const nuevosPares = pares.map(([nombre, precio]) => {
  return [nombre, precio * 2];
});
const actualizados = Object.fromEntries(nuevosPares);
console.log(actualizados); // { libro: 40, lapiz: 4 }

const porPrecio = [...pares].sort((a, b) => a[1] - b[1]);
console.log(porPrecio); // [["lapiz", 2], ["libro", 20]]
```

fromEntries construye un objeto a partir de pares [clave, valor]. Si una clave se repite, conserva el último valor.

No transforma cualquier array arbitrario en un objeto: necesita entradas con clave y valor. Para presentar un orden de negocio, conserva un array ordenado de entradas en vez de depender del orden de propiedades de un objeto.

Elegir el método según el resultado

Una acción por elemento: forEach → undefined.
Un valor transformado por elemento: map → array nuevo.
Los elementos que cumplen una condición: filter → array nuevo.
La primera coincidencia: find → elemento o undefined.
La posición de la primera coincidencia: findIndex → índice o -1.
Una comprobación: some / every → booleano.
Un resultado acumulado: reduce → valor acumulado.
Ordenar: sort → el mismo array, modificado.

Antes de escribir una cadena de métodos, explica qué datos entran y qué resultado necesitas. Prefiere pasos intermedios con nombres claros cuando una expresión larga resulte difícil de entender.

Módulos: separar responsabilidades

Un módulo expone valores con export y utiliza valores de otros archivos con import. Sus declaraciones tienen ámbito de módulo.

utilidades.js

```
export const doble = n => n * 2;
export const nombreCurso = "JavaScript";
```

app.js

```
import { doble, nombreCurso } from "./utilidades.js";
console.log(nombreCurso, doble(4)); // "JavaScript" 8
```

Sustituye el script del HTML inicial

```
<script type="module" src="app.js"></script>
```

Abre index.html con Live Server, mediante HTTP, y utiliza rutas relativas con extensión .js. Los módulos del navegador se difieren automáticamente. No hace falta añadir otro script para utilidades.js.

Exportaciones nombradas y default

datos.js

```
const curso = { titulo: "JavaScript" };
export const horas = 20;
export default curso;
```

app.js

```
import cursoActual, { horas as duracion } from "./datos.js";
console.log(cursoActual.titulo, duracion);
// "JavaScript" 20
```

Las exportaciones nombradas se importan entre llaves; puedes renombrarlas con as. Una exportación default se importa sin llaves y puede recibir otro nombre local.

Solo puede haber un default por módulo. No es válido export default let curso = ...: declara la variable y después expórtala. Importar un nombre concreto no significa que el navegador descargue únicamente esa función.

Errores habituales con módulos

Cannot use import statement outside a module: comprueba type="module".
404 al cargar un archivo: comprueba ruta, extensión y mayúsculas.
Exportación no encontrada: revisa el nombre exportado y las llaves.
Bloqueo al abrir un archivo local: usa Live Server en lugar de file://.

Alternativa para app.js: usa utilidades.js de la página 27

```
import * as utilidades from "./utilidades.js";
console.log(utilidades.doble(5)); // 10
```

La importación estática se escribe en el nivel superior del módulo. La eliminación de código no usado en producción depende de herramientas de construcción; no es una consecuencia automática de importar entre llaves.

PRÁCTICA GUIADA

Práctica guiada · Analizar un catálogo

Objetivo: separar datos, funciones de análisis y programa principal.

A partir de tres productos, obtener:

- Los nombres de los productos con stock, ordenados por precio.
- El valor total del inventario: suma de precio × stock.
- El producto con un id concreto o un mensaje de ausencia.

Crea productos.js, analisis.js y app.js en la misma carpeta. Utiliza el HTML de inicio con type="module". Las tres páginas siguientes contienen los archivos completos. La salida aparece en la consola; todavía no utilizamos DOM.

PRÁCTICA GUIADA

1 · Datos del catálogo

productos.js

```
export const productos = [
  { id: 1, nombre: "Libro", precio: 20, stock: 3 },
  { id: 2, nombre: "Lápiz", precio: 2, stock: 0 },
  { id: 3, nombre: "Cuaderno", precio: 5, stock: 4 }
];
```

Cada producto tiene identificador, nombre, precio y stock. El stock 0 es válido y significa que no quedan unidades.

Datos de partida conocidos: id es único; precio y stock son números no negativos. Valor del inventario: 20 × 3 + 2 × 0 + 5 × 4 = 80. No incluimos validación de formularios en esta práctica.

PRÁCTICA GUIADA

2 · Funciones de análisis

analisis.js

```
export function nombresDisponibles(datos) {
  return datos
    .filter(p => p.stock > 0)
    .sort((a, b) => a.precio - b.precio)
    .map(p => p.nombre);
}
export function valorInventario(datos) {
  return datos.reduce((total, { precio, stock }) => {
    return total + precio * stock;
  }, 0);
}
export function buscarProducto(datos, id) {
  return datos.find(p => p.id === id);
}
```

filter crea un array nuevo; sort ordena ese resultado, sin reordenar datos. Las funciones no modifican las propiedades de los productos. reduce utiliza 0 como valor inicial y find puede devolver undefined.

PRÁCTICA GUIADA

3 · Programa principal y resultados

app.js

```
import { productos } from "./productos.js";
import {
  nombresDisponibles, valorInventario, buscarProducto
} from "./analisis.js";

console.log(nombresDisponibles(productos));
// ["Cuaderno", "Libro"]
console.log(valorInventario(productos)); // 80

const encontrado = buscarProducto(productos, 9);
console.log(encontrado ?? "Producto no encontrado");
// "Producto no encontrado"
console.log(nombresDisponibles([])); // []
console.log(valorInventario([])); // 0
```

Comprueba además el id 1: debe devolver el objeto Libro. Cambia el stock del Lápiz a 2: los nombres serán Lápiz, Cuaderno y Libro; el valor del inventario será 84. Después restaura los datos originales.

Ejercicios para clase

1 · Datos ausentes. Una configuración tiene volumen 0, nombre "" y tema null. Aplica ?? y explica qué valores se conservan.

2 · Funciones. Escribe una flecha que devuelva el triple y una función aplicar que reciba un número y una callback. Añade una función sumar con rest.

3 · Colecciones. Con [4, 7, 9, 3], obtiene aprobados, primera nota suspensa, suma y copia ordenada. El array original debe conservarse.

4 · Objetos y módulos. Duplica los valores de { Ana: 8, Luis: 6 } con entries, map y fromEntries. Exporta la función e impórtala desde app.js.

Comprueba entradas vacías, valores 0 y búsquedas sin resultados. Entrega archivos y resultados de prueba. Ampliación: añade al catálogo una función que devuelva el producto más barato con stock sin modificar los datos.

Resumen y documentación

Ideas clave: ?? trata datos ausentes; las flechas necesitan un retorno correcto; rest reúne y spread expande; las copias pueden compartir objetos; cada método de array devuelve un tipo de resultado; los módulos organizan dependencias.

Referencias técnicas de MDN

Valores por defecto · Nullish coalescing
Funciones flecha · Rest
Spread · Desestructuración
Arrays · Object
Módulos JavaScript

Adaptación didáctica del material aportado: Características avanzadas de JavaScript, Daniel López Lozano. Ejemplos revisados y rediseño: Antonio Martín León. Consulta técnica: 24/09/2026. Siguiente unidad: Eventos y DOM.